

Proyecto PanComido

Resumen de Arquitectura Aplicada

Este documento detalla la estructura de carpetas del frontend en Angular 21+. La arquitectura implementa un diseño modular orientado a dominios (Features), manteniendo una separación clara de responsabilidades.

Cambios Principales Aplicados

X	RxJS y Store complejos descartados: Se eliminó el uso de NgRx/BehaviorSubjects a favor de Signals nativos.
V	Lógica en State Services: La lógica de negocio ahora reside en servicios de estado por feature (Equivalente a Use Cases).
V	Entidades anémicas: Los modelos del frontend (Interfaces) son solo contenedores de datos.
V	Unificación de capas UI: División estricta en <i>Smart Components</i> (Containers) y <i>Dumb Components</i> (Presentacionales).
V	Componentes reutilizables: Centralizados en la capa Shared/UI, 100% aislados del dominio.

Estructura Completa de Carpetas

```
frontend/
+-- angular.json
+-- src/
    +-- main.ts
    +-- app.config.ts          <- CONFIGURACIÓN (Inyección de
dependencias global)
    +-- app.routes.ts         <- RUTAS RAÍZ (Lazy loading por
roles)
    +-- app/
        +-- core/             <- NÚCLEO GLOBAL (Singletons)
            | +-- guards/
            | | +-- auth.guard.ts
            | | +-- role.guard.ts
            | +-- interceptors/
            | | +-- auth.interceptor.ts
            | +-- layouts/     <- INFRAESTRUCTURA VISUAL DE RUTAS
            | | +-- admin-layout/
            | | +-- auth-layout/
            | +-- models/
            | | +-- pedido.model.ts <- Entidades ANÉMICAS (solo datos
tipados)
            | | +-- producto.model.ts
            | | +-- enums.ts     <- Objetos de valor inmutables
(Roles, Estados)
            | +-- services/
            | +-- api.service.ts <- Cliente HTTP Base (Reutilizable)
            |
        +-- shared/           <- UI KIT (Cero lógica de negocio)
            | +-- directives/
            | | +-- click-outside.directive.ts
            | +-- pipes/
            | +-- ui/          <- COMPONENTES TONTOS GLOBALES
(Dumb)
            | +-- boton/
            | +-- buscador/
            | +-- dropdown/
            | +-- page-toolbar/
```

Estructura Completa de Carpetas (Continuación)

```
+-- features/                                <- MÓDULOS DE DOMINIO (Independientes)
|
|   +-- stock/                               <- FEATURE: STOCK / INVENTARIO
|   |   +-- containers/                     <- SMART COMPONENTS (Conectan
estado con vista)
|   |   |   +-- stock-page.ts
|   |   |   +-- components/                 <- DUMB COMPONENTS LOCALES (UI
específica)
|   |   |   +-- stock-list.ts
|   |   |   +-- services/
|   |   |   +-- stock.api.ts               <- CAPA DE ACCESO A DATOS
(Equivalente al Repository)
|   |   |   +-- stock.state.ts             <- LÓGICA DE NEGOCIO (Equivalente
al Use Case)
|   |   +-- stock.routes.ts
|   |
|   +-- pedidos/                             <- FEATURE: PEDIDOS Y MESAS
|   |   +-- containers/
|   |   |   +-- pedidos-page.ts
|   |   |   +-- components/
|   |   |   +-- pedido-card.ts
|   |   |   +-- services/
|   |   |   +-- pedido.api.ts
|   |   |   +-- pedido.state.ts
|   |   +-- pedidos.routes.ts
```

Descripción Detallada de Capas

1. Capa Core (Núcleo)

Responsabilidad: Elementos estructurales que sostienen la aplicación entera. Son instanciados una sola vez (Singletons) o configuran la aplicación. NO dependen de ningún dominio de negocio específico, sino que proveen las herramientas para que los dominios funcionen.

Directorio	Contenido y Reglas
Models	Entidades anémicas (Interfaces TS). Representan los DTOs exactos que envía y recibe el backend.
Guards	Middlewares de seguridad de Angular para impedir acceso a rutas no autorizadas.
Layouts	Estructura base de las páginas (Sidebar + Header + RouterOutlet). Van aquí porque están acoplados directamente al árbol de rutas.
Services	Cliente HTTP genérico y estado global del usuario logueado.

2. Capa Shared (UI Compartida)

Responsabilidad: Proporcionar la librería de componentes visuales (Design System). Los componentes aquí son 100% "tontos" (Dumb Components). **Cero lógica de negocio.**

Directorio	Contenido y Reglas
UI / Components	Botones, modales, toolbars. Solo usan input() y output(). Jamás inyectan un servicio ni conocen modelos como Pedido o Producto.

Directives / Pipes	Herramientas de formateo o manipulación de DOM (Ej: formatear moneda, detectar clics afuera).
---------------------------	---

Descripción Detallada de Capas (Continuación)

3. Capa Features (Dominios de Negocio)

Responsabilidad: Implementar los casos de uso reales de la aplicación. Todo el código que resuelve problemas de PanComido vive aquí, dividido en silos (módulos) verticales.

Directorio	Contenido y Reglas
Containers (Smart)	Páginas y modales inteligentes. Inyectan los State Services, leen Signals y delegan datos al HTML. Orquestan eventos.
Components (Dumb)	Componentes visuales específicos de este dominio. (Ej: stock-list). Se alimentan del Container.
API Services	Equivalente a los <i>Repositories</i> del backend. Realizan llamadas HttpClient puras. Sin estado.
State Services	Equivalente a los <i>Use Cases</i> del backend. Mantienen los Signals (memoria local) y orquestan validaciones y llamadas a la API.

Flujo de Datos en la Aplicación

Flujo completo de una petición en el frontend, manteniendo el flujo unidireccional y reactivo:

1. [USUARIO] hace clic en el botón "Crear Producto" en la UI.

```

    |
    v
2. [DUMB COMPONENT] (Ej. app-boton)
   Emite el evento sin procesar lógica: (clicked)="emitir()"
   |
   v
3. [SMART COMPONENT] (Ej. stock-page.ts)
   Captura el evento HTML y delega al servicio:
   this.stockState.crearProducto(datosFormulario)
   |
   v
4. [STATE SERVICE] (El "Use Case" del Frontend)
   - Mutación local: this._loading.set(true)
   - Llama al repositorio (API Service)
   |
   v
5. [API SERVICE] (El "Repository" del Frontend)
   - Ejecuta HTTP POST /api/stock
   |
   | (Viaje de red hacia el Backend -> Controller -> UseCase -> DB)
   |
   v
6. [API SERVICE] recibe la respuesta (ProductoResponseDto)
   |
   v
7. [STATE SERVICE] procesa la respuesta
   - Mutación local: this._productos.update(p => [...p, nuevoProducto])
   - Mutación local: this._loading.set(false)
   |
   v
8. [SIGNALS Y COMPONENTES]
   El Smart Component detecta el cambio en la Signal de forma automática.
   Angular aplica `OnPush` y re-renderiza eficientemente solo las
   partes del DOM afectadas.

```

Concepto: Entidades Anémicas (Interfaces)

Las entidades de dominio en el frontend son estrictamente contenedores de datos tipados (Interfaces en TypeScript). **NO contienen lógica de negocio, métodos ni validaciones.** Son el espejo exacto de los DTOs que provienen del backend.

```
// core/models/pedido.model.ts (ANÉMICA)

export interface Pedido {
  id: number;
  fechaCreacion: string;
  estado: EstadoPedido;
  total: number;
  detalles: DetallePedido[];
}

export interface DetallePedido {
  productoId: number;
  cantidad: number;
  precioUnitario: number;
  subtotal: number;
}

// ❌ MAL: No crear clases con constructores o métodos (Ej.
  calcularTotal())
// Toda esa lógica pertenece al State Service (Use Case) o llega resuelta
// desde el backend.
```

Concepto: State Services (Casos de Uso)

Los State Services contienen **TODA** la lógica de estado local y orquestación del frontend. Cada servicio de estado tiene una responsabilidad única sobre un dominio y reemplaza a los antiguos stores complejos (como NgRx).

```
// features/stock/services/stock.state.service.ts
import { Injectable, inject, signal, computed } from '@angular/core';
import { StockApiService } from '../stock.api.service';
import { ProductoStock } from 'src/app/core/models/stock.model';

@Injectable({ providedIn: 'root' })
export class StockStateService {
  private api = inject(StockApiService);

  // 1. Estado PRIVADO (Solo este servicio puede mutarlo)
  private _productos = signal<ProductoStock[]>([]);
  private _loading = signal<boolean>(false);

  // 2. Estado PÚBLICO (Componentes consumen esto en solo lectura)
  productos = this._productos.asReadonly();
  loading = this._loading.asReadonly();

  // 3. Variables Derivadas (Computed)
  productosCriticos = computed(() =>
    this._productos().filter(p => p.stock < p.stockMinimo)
  );

  // 4. MÉTODOS DE NEGOCIO (Equivalentes a UseCases)
  cargarStock(): void {
    this._loading.set(true);
    this.api.getAll().subscribe({
      next: (data) => {
        this._productos.set(data);
        this._loading.set(false);
      },
    });
  }
}
```



```

        error: () => this._loading.set(false)
    });
}

crearProducto(datos: any): void {
    // Lógica para enviar al API y actualizar el Signal...
}
}

```

Concepto: Smart vs Dumb Components

Para lograr la separación de responsabilidades en la UI, dividimos la capa de presentación en dos tipos de componentes.

1. Smart Component (Container)

Es el "pegamento". Inyecta el State Service, expone los Signals al HTML y captura eventos de los componentes hijos.

```

// features/stock/containers/stock-page.component.ts
@Component({
    selector: 'app-stock-page',
    template: `
        <app-page-toolbar>
            <app-boton (clicked)="abrirModal()">Nuevo</app-boton>
        </app-page-toolbar>

        <!-- Pasa el Signal directamente usando () -->
        <app-stock-list
            [productos]="stockState.productos()"
            (borrar)="eliminarItem($event)">
        </app-stock-list>
    `
})

```

```

export class StockPageComponent {
  stockState = inject(StockStateService); // Inyección de lógica

  ngOnInit() {
    this.stockState.cargarStock();
  }

  eliminarItem(id: number) {
    this.stockState.eliminar(id);
  }
}

```

2. Dumb Component (Presentacional)

No tiene idea del negocio. Recibe data por input() y avisa de eventos por output().

```

// features/stock/components/stock-list.component.ts
@Component({
  selector: 'app-stock-list',
  template: `
    @for(item of productos(); track item.id) {
      <div>{{ item.nombre }} - <button
(click)="borrar.emit(item.id)">X</button></div>
    }
  `
})
export class StockListComponent {
  // Angular 21+: Signals Inputs
  productos = input<ProductoStock[]>([]);
  borrar = output<number>();
}

```

Guía de Decisiones (Qué crear y dónde)

Para evitar mezclar capas, antes de generar un componente, evalúa esta matriz:

Necesidad	Tipo a crear	Ubicación exacta
Crear un botón, un buscador o un componente genérico de UI (reutilizable).	Dumb Component	shared/ui/nombre/
Crear el contenedor general (layout) para el mozo (sidebar + header).	Layout Component	core/layouts/mozo-layout/
Crear una vista principal para listar pedidos en pantalla.	Smart Component	features/pedidos/containers/pedidos-page/
Crear una tabla/lista específica que solo muestra pedidos.	Dumb Component	features/pedidos/components/pedidos-list/
Crear lógica para calcular el total local y hacer un GET HTTP.	State / Api Service	features/pedidos/services/

Generación de Componentes (Angular CLI)

Para agilizar la creación estructural, usar la consola dentro del directorio base respetando las banderas (Standalones y OnPush por defecto):

```
# Para crear un Dumb Component en Shared UI
ng g c shared/ui/dropdown --standalone -c OnPush
```

```
# Para crear un Smart Component en una feature
ng g c features/stock/containers/stock-page --standalone -c OnPush
```

```
# Para crear los servicios de dominio (equivalente a scaffolding)
```

```
ng g s features/stock/services/stock.state
ng g s features/stock/services/stock.api
```

Estrategia de Testing

La estrategia de testing asegura la lógica de forma aislada, evitando montar el DOM cuando no es necesario. (TEST EN DOMINIO = TEST EN STATE SERVICE).

State Services (Lógica)	Tests unitarios inyectando espías (Mocks) en el ApiService. Validar que los Signals cambien correctamente tras recibir respuestas ficticias HTTP.
Smart Components	Shallow testing (sin renderizar hijos). Mockear el State Service y probar que al disparar un evento UI, se llame al método del servicio.
Dumb Components	Testing visual básico. Verificar que un input inyectado renderice el HTML esperado y que el botón emita un output.

```
// PanComido.Frontend.Tests/features/stock/stock.state.service.spec.ts
```

```
describe('StockStateService', () => {
  let service: StockStateService;
  let mockApi: jasmine.SpyObj<StockApiService>;

  beforeEach(() => {
    mockApi = jasmine.createSpyObj('StockApiService', ['getAll']);
    TestBed.configureTestingModule({
      providers: [
        StockStateService,
        { provide: StockApiService, useValue: mockApi }
      ]
    });
  });
});
```

```

    });

    service = TestBed.inject(StockStateService);
  });

  it('Debe cargar stock y actualizar signals', () => {
    const mockData = [{ id: 1, nombre: 'Harina', stock: 10 }];
    mockApi.getAll.and.returnValue(of(mockData)); // Simular respuesta

    service.cargarStock(); // Ejecutar Acción

    expect(service.loading()).toBeFalse(); // Verificar estado final
    expect(service.productos()).toEqual(mockData); // Verificar mutación
  });
});

```

Configuración de Dependency Injection (app.config.ts)

Este archivo es el equivalente exacto al Program.cs del backend. Centraliza el arranque, el cliente HTTP y el enrutador. Angular usa inyección de dependencias jerárquica (Tree-shakable).

```

// src/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    // 1. Configuración de Rutas base
    provideRouter(routes),

```

```
// 2. Cliente HTTP y Middlewares (Interceptores)
provideHttpClient(
  withInterceptors([authInterceptor])
)

// 3. Notas adicionales:
// Los servicios (State y Api) no necesitan listarse aquí
// gracias al decorador @Injectable({ providedIn: 'root' })
// lo cual habilita el tree-shaking nativo de Angular.
]
};
```

Resumen y Próximos Pasos

Ventajas de esta arquitectura frontend:

- **V Separación clara de responsabilidades:** UI tonta vs Lógica encapsulada en contenedores y servicios.
- **V Testeable:** La lógica de UI (State Services) se puede probar sin montar el DOM en absoluto.
- **V Mantenable:** Cada módulo funcional tiene su propio ecosistema. Es fácil de escalar.
- **V Sincronía con Backend:** El lenguaje, los modelos y las carpetas espejan el DDD del servidor, mejorando la comunicación Full-Stack.
- **V Máximo Rendimiento:** El uso de Signals + OnPush evita la carga excesiva en dispositivos táctiles (Pos / Tablets del local).

Plan de acción recomendado:

1. Configurar app.config.ts con provideHttpClient.
2. Crear los modelos anémicos en core/models/ copiando los DTOs C#.

3. Construir la librería de UI en `shared/ui/` usando Content Projection (Toolbar, Botones).
4. Generar los Layouts Base en `core/layouts/` (Dashboard, Auth).
5. Generar *Features*: `features/stock/`, `features/pedidos/`, etc.
6. Implementar State y API services para cada Feature.
7. Maquetar los contenedores y componentes locales en cada Feature.
8. Acoplar Rutas (`app.routes.ts`) con Lazy Loading.
9. Escribir Tests unitarios aislados para los State Services de cada dominio.